Universidad de Chile
Facultad de Ciencias Físicas y Matemáticas
Departamento de Ingeniería Eléctrica

[PENDING-TITLE].

[NO-CODE] [PENDING-SUBTITLE]

Integrantes: Diego Peña Loza
Roberto Guajardo M.

Profesor: Constanza Ahumada
Encargados de Laboratorio: [PENDING].

Santiago de Chile

Índice de Contenidos

1. Introducción **1**

2. Setup físico **1**
2.1. Descripción del setup físico . . . 1
2.2. Componentes del setup . . . 1

3. Código SDCMotor **2**
3.1. Propósito y alcance . . . 2
3.1.1. Red y configuración fija en el código . . . 2
3.1.2. Requisitos de compilación (MEX/S-Function) . . . 2
3.1.3. Compilación recomendada desde MATLAB . . . 2
3.2. Dependencias y compilación . . . 3
3.3. Parámetros del bloque . . . 3
3.4. Interfaz de entradas y salidas . . . 3
3.5. Vectores de trabajo y estados . . . 3
3.6. Manejo de errores . . . 4
3.7. Ciclo de vida del bloque (funciones S-Function) . . . 4
3.7.1. `mdlInitializeSizes(S)`: declaración de capacidades . . . 4
3.7.2. `mdlInitializeSampleTimes(S)`: tiempo de muestreo . . . 4
3.7.3. `mdlStart(S)`: apertura de conexión con OPTO22 . . . 4
3.7.4. `mdlUpdate(S,tid)`: escritura de comandos (salida hacia planta) . . . 5
3.7.5. `mdlOutputs(S,tid)`: lectura de mediciones (entrada desde planta) . . . 5
3.7.6. `mdlTerminate(S)`: cierre seguro . . . 5
3.8. Resumen operacional (pseudoflujo) . . . 5
3.9. Consideraciones de integración y configuración . . . 7

4. Consideraciones y Precauciones **8**
4.1. Consideraciones y precauciones hardware . . . 8
4.2. Consideraciones Simulación en Tiempo Real Simulink . . . 8
4.3. Consideraciones y precauciones módulos opto 22 . . . 8
4.3.1. Módulos de E/S utilizados . . . 8
4.3.2. Límites eléctricos y protección del hardware OPTO-22 . . . 8
4.3.3. Conexión de Red . . . 8

5. Sensor de corriente **10**
5.1. Placa de medición de corriente . . . 10

6. Carga electronica **13**
6.1. Carga electrónica . . . 13

Referencias **15**

Índice de Figuras

1. Caption 6
2. Acondicionamiento de la señal de corriente 10
3. Acondicionamiento señal Arduino (sin uso con Opto-22) 11
4. Fuente simétrica 11
5. Esquemático completo 12
6. Esquemático carga electrónica 13

Índice de Tablas

1. Mapa de puertos de Simulink a puntos analógicos OPTO22. 3
2. Temperaturas medidas 14

1. Introducción

Este experimento consta de un motor DC, una carga electrónica, un sensor de corriente y un sensor de velocidad.

2. Setup físico

2.1. Descripción del setup físico

2.2. Componentes del setup

3. Código SDCMotor

3.1. Propósito y alcance

El archivo `SDCMotor.cpp` implementa una *S-Function Level-2* para Simulink cuyo objetivo es actuar como puente entre el modelo de control y un módulo de E/S analógicas OPTO22 (vía Ethernet), utilizando la librería `O22SIOMM` (clase `O22SnapIoMemMap`). En cada paso de simulación el bloque:

- **envía** al OPTO22 dos comandos analógicos: comando de voltaje para el actuador, y una corriente de carga (`iLoad`)[1];
- **recibe** desde el OPTO22 dos mediciones analógicas: velocidad (`velocidad`) y corriente (`corriente`);
- **detiene** la simulación si se acumulan demasiados errores de comunicación.

3.1.1. Red y configuración fija en el código

La S-Function abre la comunicación Ethernet con un *Brain* OPTO22 en la IP `192.168.6.101` y puerto `2001`. Por lo tanto, el PC y el equipo OPTO22 deben estar en la misma red/subred y sin bloqueo por firewall/ACL. (Estos valores están embebidos en `SDCMotor.cpp`; si se cambia el setup, deben actualizarse en el código o parametrizarse.)

3.1.2. Requisitos de compilación (MEX/S-Function)

Para generar el binario `SDCMotor.mex*` se requiere:

- **Un compilador C/C++ soportado por MATLAB** configurado con `mex -setup C++`.
- **Headers y librería** de OPTO22 (`O22SIOMM.h` y la correspondiente `.lib/.dll` o fuentes) disponibles en rutas conocidas.

MATLAB permite especificar rutas de *include* con `-I` y enlazado con opciones de librería durante `mex`.

3.1.3. Compilación recomendada desde MATLAB

Desde la carpeta donde reside `SDCMotor.cpp`, compilar preferentemente con:

```
mex -v SDCMotor.cpp ...
  -I"<RUTA_OPTO22>\include" ...
  -L"<RUTA_OPTO22>\lib" -lO22SIOMM
```

Si la distribución de OPTO22 entrega la implementación como fuentes (por ejemplo, `O22SIOMM.cpp`), una alternativa directa es:

```
mex -v SDCMotor.cpp O22SIOMM.cpp -I"<RUTA_OPTO22>\include"
```

Tras compilar, el archivo `SDCMotor.mex*` debe quedar en el *MATLAB path* o en el mismo directorio del modelo para que Simulink pueda cargar el bloque.

[1] El significado físico exacto depende de la conexión real.

3.2. Dependencias y compilación

- **API Simulink:** `simstruc.h` para la definición de S-Functions.
- **OPTO22:** `O22SIOMM.h` y la implementación asociada de la librería (debe estar disponible al compilar el MEX).

La S-Function se declara como:

```
#define S_FUNCTION_NAME  SDCMotor
#define S_FUNCTION_LEVEL 2
```

y se encapsula en `extern C` para asegurar *linkage* compatible con el mecanismo de carga de Simulink.

3.3. Parámetros del bloque

El bloque define **un (1) parámetro** de máscara:

- **P1:** tiempo de muestreo T_s (segundos). Se lee en `mdlInitializeSampleTimes()` mediante:

$$T_s \leftarrow \texttt{mxGetScalar(ssGetSFcnParam(S,0))}$$

Si la cantidad de parámetros provistos en Simulink no coincide con lo esperado, `mdlInitializeSizes()` retorna y Simulink reporta el error.

3.4. Interfaz de entradas y salidas

El bloque tiene **2 entradas escalares** y **2 salidas escalares**, todas tipo `real_T`:

Tabla 1: Mapa de puertos de Simulink a puntos analógicos OPTO22.

Puerto	Señal	Descripción
Entrada 0	u	Comando de voltaje a enviar al OPTO22 (canal analógico 0).
Entrada 1	$iLoad$	Corriente de carga a enviar al OPTO22 (canal analógico 1).
Salida 0	y	Velocidad leída desde OPTO22 (canal analógico 4).
Salida 1	z	Corriente leída desde OPTO22 (canal analógico 6).

Además, se exige memoria contigua para ambas entradas (`ssSetInputPortRequiredContiguous`), lo que simplifica el acceso a las señales.

3.5. Vectores de trabajo y estados

Para mantener información entre llamados, la S-Function reserva:

- **PWork (1 puntero):** almacena un puntero a la instancia `O22SnapIoMemMap` que representa la conexión activa con el OPTO22.
- **IWork (1 entero):** contador de errores de comunicación (`WORK_INT_ERROR_COUNT`).

Se define además **1 estado discreto** (`ssSetNumDiscStates(S,1)`), aunque en esta implementación no se observa uso explícito de `ssGetX`/`ssSetX`; en la práctica, la memoria persistente relevante queda cubierta por `PWork` e `IWork`.

3.6. Manejo de errores

El bloque implementa un mecanismo simple de tolerancia a fallos:

- Cada vez que una operación de lectura/escritura hacia OPTO22 retorna un código distinto de `SIOMM_OK`, se incrementa `IWork[ERROR_COUNT]` y se imprime un mensaje vía `ssPrintf`.
- Si el contador supera `MAX_ERROR_COUNT` (definido como 5), se invoca `ssSetErrorStatus(S, "...")` y Simulink detiene la ejecución del modelo.

3.7. Ciclo de vida del bloque (funciones S-Function)

A continuación se describe el rol de cada función obligatoria/registrada y el flujo típico en simulación.

3.7.1. mdlInitializeSizes(S): declaración de capacidades

Configura la firma del bloque: número de parámetros, puertos de entrada/salida, tiempos de muestreo y tamaños de vectores de trabajo. En particular:

- 1 parámetro esperado.
- 2 entradas escalares y 2 salidas escalares.
- 1 tiempo de muestreo.
- `PWork` para el objeto de interfaz OPTO22 y `IWork` para el contador de errores.

3.7.2. mdlInitializeSampleTimes(S): tiempo de muestreo

Asigna el tiempo de muestreo del bloque a partir del parámetro T_s y fija offset en 0:

$$T_s = \texttt{Param(0)}, \qquad \text{offset} = 0.$$

3.7.3. mdlStart(S): apertura de conexión con OPTO22

Se ejecuta una sola vez al inicio. Sus pasos son:

1. Crea el objeto `Brain = new O22SnapIoMemMap()`.
2. Abre conexión Ethernet con:
   - IP: `192.168.6.101`
   - Puerto: `2001`
   - Timeout: `10000` (ms)
   - Último argumento: `1` (dependiente de la API; típicamente flags/modo).
3. Espera a que la apertura finalice consultando `IsOpenDone()` mientras retorne `SIOMM_ERROR_NOT_CONNECTED_YET`.
4. Si ocurre un error, define un estado de error fatal:

`Ünable to connect to OPTO22 (192.168.6.101)."`

5. Si la conexión es exitosa, guarda el puntero del objeto en `ssGetPWork(S)[0]` e inicializa el contador de errores en 0.

3.7.4. mdlUpdate(S,tid): escritura de comandos (salida hacia planta)

En cada paso de simulación, el bloque toma las entradas de Simulink y las envía al OPTO22:

- Entrada 0 (u) → `SetAnaPtValue(0, (float)*u)`.
- Entrada 1 ($iLoad$) → `SetAnaPtValue(1, (float)*iLoad)`.

Cada envío es verificado; ante error se incrementa el contador y, si se supera el umbral, se detiene la simulación.

Nota.

El mensaje impreso ante error en el segundo envío repite el texto `Error sending voltage command.\n`; si se quiere mayor trazabilidad, se puede diferenciar mensajes (p.ej. "voltage" vs "load current").

3.7.5. mdlOutputs(S,tid): lectura de mediciones (entrada desde planta)

En cada paso, el bloque consulta dos valores analógicos:

- Velocidad: `GetAnaPtValue(4, &velocidad)` → $y =$ `(real_T)velocidad`.
- Corriente: `GetAnaPtValue(6, &corriente)` → $z =$ `(real_T)corriente`.

Las lecturas tienen la misma política de manejo de errores descrita previamente.

3.7.6. mdlTerminate(S): cierre seguro

Al terminar la simulación:

1. Recupera el objeto desde `PWork`.
2. Envía **0.0** al canal analógico 0 (`SetAnaPtValue(0,0.0)`) como acción de seguridad para dejar el comando de voltaje en cero.
3. **AGREGAR: Brain->SetAnaPtValue(1,0.0); para cierre seguro del nuevo canal!!!!!!!**
4. Cierra la conexión (`Close()`).
5. Libera memoria (`delete Brain`).

3.8. Resumen operacional (pseudoflujo)

```
Start:
  conectar a OPTO22 (IP fija)
  error_count = 0

Cada Ts:
```

```
  mdlUpdate:
    enviar u -> AO0
    enviar iLoad -> AO1
  mdlOutputs:
    leer velocidad <- AI4
    leer corriente <- AI6
  si error_count > 5:
    detener simulación

Terminate:
  AO0 = 0
  AO1 = 0
  cerrar conexión
```

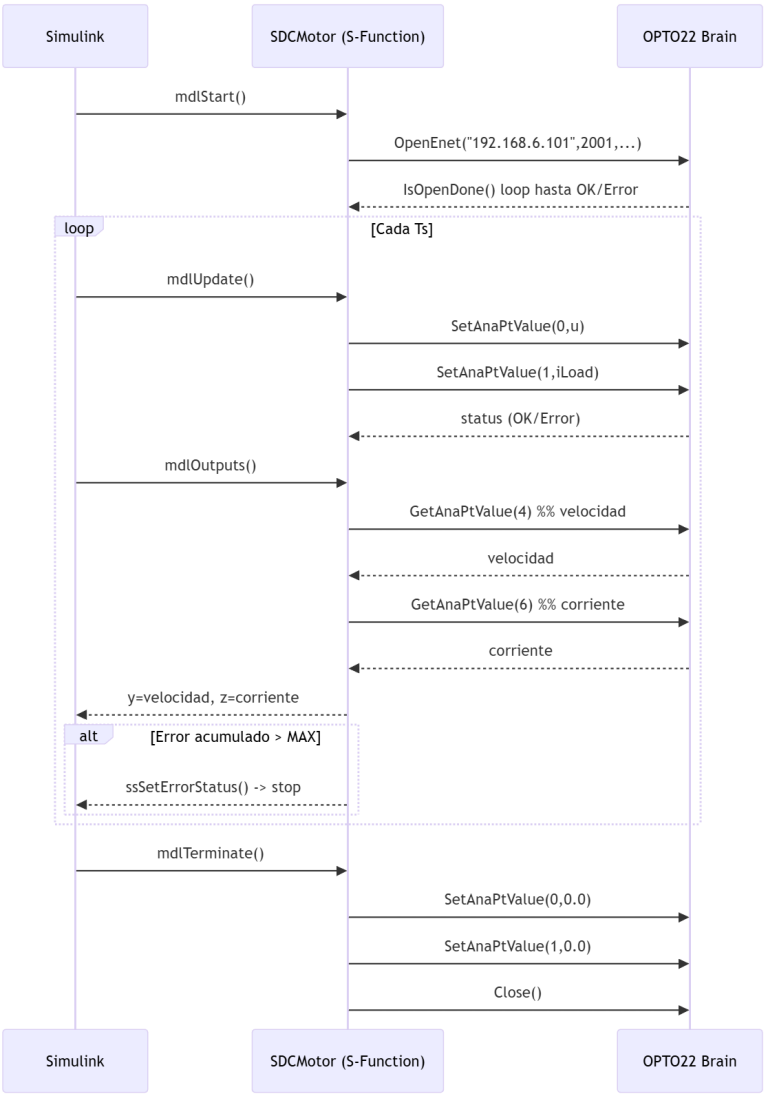

Figura 1: Caption

3.9. Consideraciones de integración y configuración

- **IP de computador HOST: PONER ESAS COSAS DE LA IP DEL PC ESPECIAL**
- **IP fija en el código:** la dirección `192.168.6.101` está embebida. Para reutilización en distintas configuraciones de red, es recomendable parametrizar IP/puerto como parámetros del bloque.
- **Entradas Analógicas adicionales:** En la configuración actual del setup, solo se ocupan 2 de los 4 puntos de entrada del modulo `SNAP-AIV-4` [1], pero habilitar puntos adicionales puede incrementar el overhead y prohibir el uso de tiempos de muestreo menores.
- **Mapa de canales:** los índices (0,1,4,6) asumen una asignación concreta de puntos analógicos en el OPTO22. Cualquier cambio de cableado/escala debe reflejarse en este mapa o, idealmente, parametrizarse.
- **Escalamiento físico:** este bloque transmite y recibe valores `float` sin aplicar conversión (p.ej., V↔RPM). Si se requiere trazabilidad metrológica, conviene documentar (y/o implementar) funciones de escala y saturación directamente en la simulación.
- **Robustez:** el umbral de errores (`MAX_ERROR_COUNT=5`) detiene la simulación. Esto es útil como medida de seguridad.

4. Consideraciones y Precauciones

4.1. Consideraciones y precauciones hardware

Cosas de la fuente corriente, máximos, peligros, etc

4.2. Consideraciones Simulación en Tiempo Real Simulink

Cosas para no laguaer, tener los tiempos de muestreo, Ts, no abusar de tantos scopes, PC tiene rendimiento limitado, entre otras cosas, evitar que el usuario manipule el bloque del motor

4.3. Consideraciones y precauciones módulos opto 22

4.3.1. Módulos de E/S utilizados

El banco de pruebas se implementa con módulos SNAP de OPTO22:

- **Dual Analog Output `SNAP-AOV-27`**: 2 canales de *salida analógica de voltaje* bipolar (rango nominal -10 a $+10$ VDC). Cada canal puede suministrar solamente ± 5 mA de corriente de carga (*no es un driver de potencia*). Ambos canales comparten referencia y **no están aislados entre sí**, aunque el módulo sí posee aislamiento respecto a otros módulos/dispositivos del rack. [2]
- **Quad Analog Input `SNAP-AIV-4`**: 4 canales de *entrada analógica de voltaje*, configurables por canal en ± 10 VDC o ± 5 VDC. Los canales **no están aislados entre sí**. [1]

4.3.2. Límites eléctricos y protección del hardware OPTO-22

Para evitar daños en los módulos y/o en la electrónica asociada, se deben respetar las siguientes restricciones:

- **No exceder el rango de salida del `SNAP-AOV-27`**: asegurar que la señal de comando enviada desde Simulink se mantenga dentro de $[-10, +10]$ V (idealmente con saturación explícita en el modelo).
- **No exigir corriente de carga mayor a ± 5 mA por canal** en el `SNAP-AOV-27`. Si se requiere accionar una etapa de potencia, debe usarse un amplificador/buffer externo de alta impedancia de entrada (p. ej. etapa de acondicionamiento) para que el módulo no entregue corriente directa a la carga.
- **Entradas del `SNAP-AIV-4`**: garantizar que las señales medidas no superen el rango configurado (± 10 V por canal).
- **Referencias/tierras**: dado que los canales no están aislados entre sí (entrada y salida), es crítico compartir referencias de señal de forma consistente y evitar lazos de tierra.

4.3.3. Conexión de Red

Con el fin de minimizar la incertidumbre temporal (*jitter*) y la latencia en el canal de comunicación, se recomienda establecer una conexión **directa** (Ethernet–RJ45) entre el *Brain* OPTO22 y el

computador que ejecuta la simulación. En lo posible, se debe evitar el uso de routers o switches intermedios, ya que introducen colas, procesamiento adicional y potenciales retransmisiones, aumentando la variabilidad del retardo entre paquetes.

Adicionalmente, se sugiere que el computador utilizado para la simulación opere en una red dedicada y, de ser viable, **sin acceso a Internet** durante la ejecución. Esto reduce tráfico de fondo (actualizaciones, sincronizaciones, servicios en segundo plano) y contribuye a mantener un canal de comunicación más estable, consistente y reproducible para el lazo de control en tiempo real.

5. Sensor de corriente

5.1. Placa de medición de corriente

Antes de todo, se menciona que los diseños de PCB fueron hecho mediante EasyEDA y la impresión mediante el lab. de electrónica. Para realizar la medición de corriente de armadura del motor, se elaboro una PCB como la que se muestra en la imagen xx,

El funcionamiento es el siguiente: la corriente de armadura del motor se mide a través de un sensor LA 55-P, este sensor, entrega una corriente que se transforma a voltaje con una resistencia y se amplifica mediante un circuito de amplificadores operacionales entregando un voltaje a la salida de la PCB, en ocasiones anteriores se necesito un arduino para digitalizar la corriente y comunicarla hacia Simulink, dado esto, fue necesario que la señal entregada por la PCB posea un offset o valor continuo de 2.5 V, y un rango de excursión de 0 a 5 V, lo que es compatible con Arduino. De esta manera, se elaboró el siguiente circuito esquemático (ver fig. 2) con amplificadores operaciones para amplificar la señal del sensor y situarla sobre un offset de 2.5 V. En el caso del sistema que usa Opto-22 como adquisidor de datos, no es necesario que exista un offset, sin embargo, este offset se puede revertir en Simulink restandole 2.5 a la señal recibida.

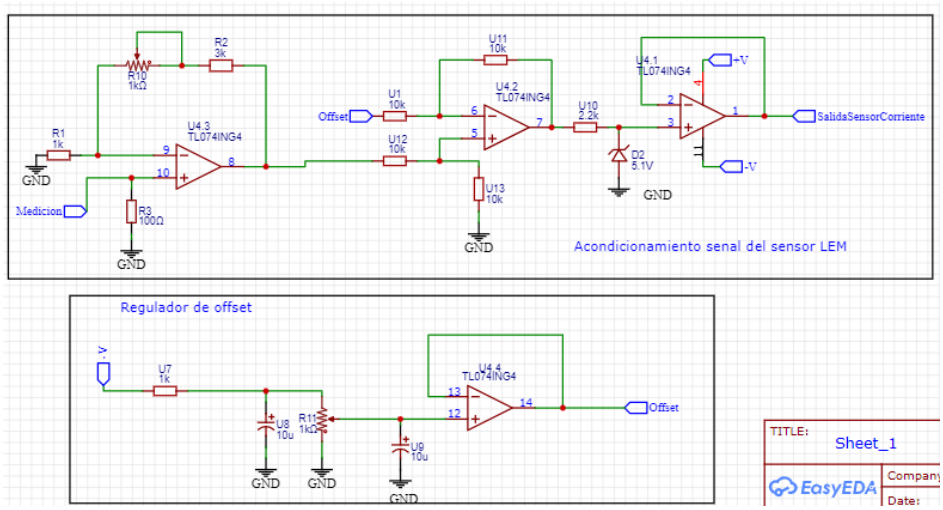

Figura 2: Acondicionamiento de la señal de corriente

Adicionalmente, existe un circuito (fig. 3) con otro amplificador operacional que se encarga de tomar una señal PWM y convertirla a un voltaje continuo además de ser amplificada por una razón de 1.5 aprox., **Esta sección del circuito NO SE USA para el montaje que utiliza el sistema Opto-22, SOLO SE USA con el montaje que utiliza un arduino como interfaz de comunicación entre el motor y Simulink**.

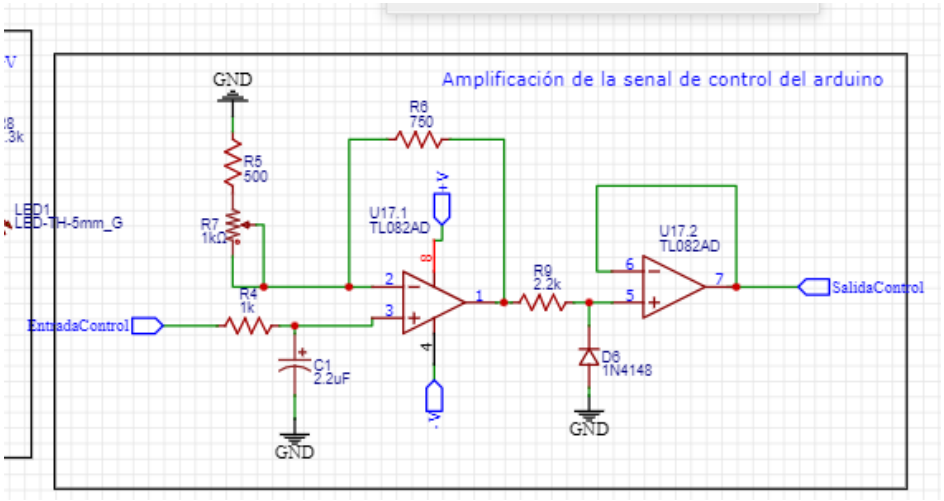

Figura 3: Acondicionamiento señal Arduino (sin uso con Opto-22)

Por ultimo, para realizar todas las funciones requeridas para acondicionar la señal de corriente, el circuito posee una fuente de alimentación simétrica como se ve en la fig. 4, el valor del voltaje de salida de la fuente es de +-12 Vdc.

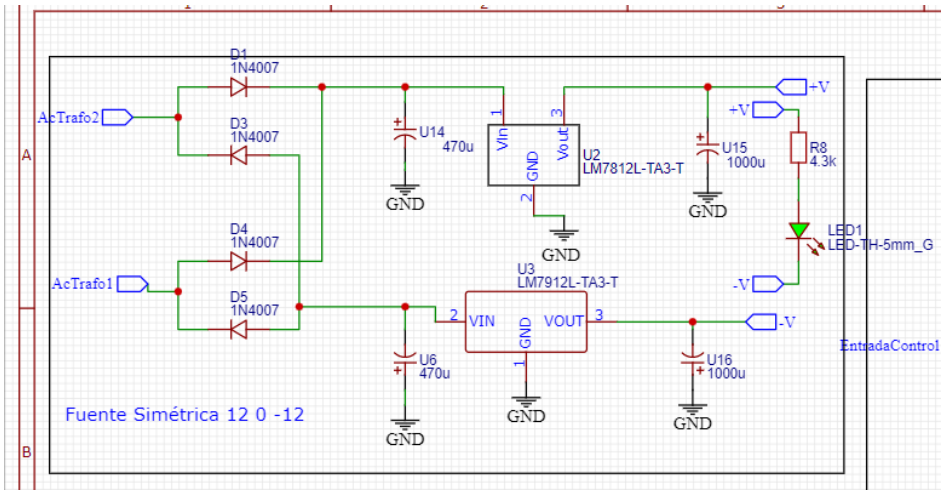

Figura 4: Fuente simétrica

Se adjunta una vista general del esquemático fig. 5.

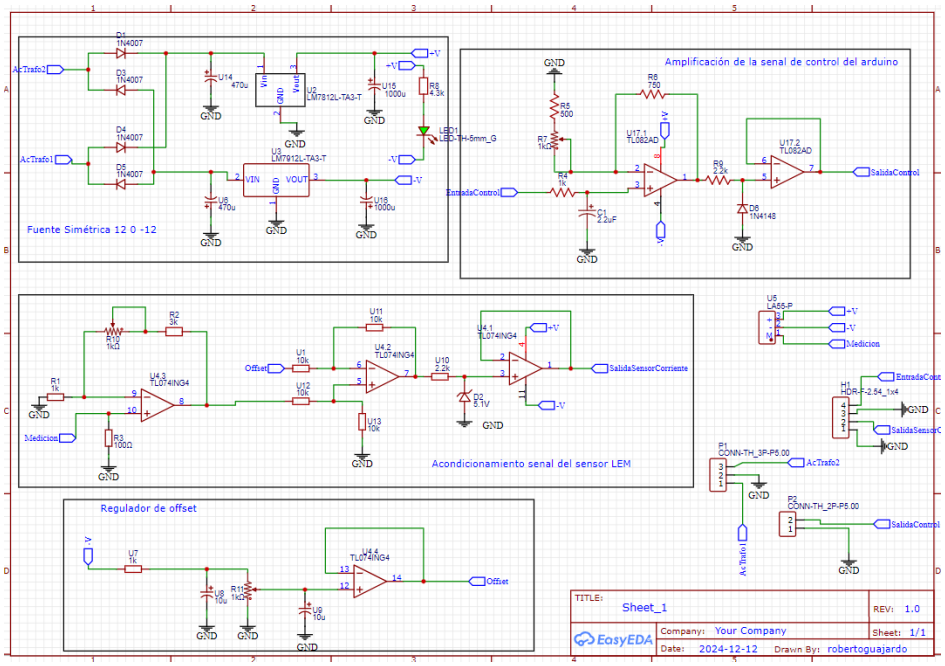

Figura 5: Esquemático completo

6. Carga electronica

6.1. Carga electrónica

En esta sección se explica el diseño y funcionamiento del circuito diseñado para actuar como una carga electrónica controlada por voltaje. El propósito de este circuito, es ofrecer una condición de carga para la armadura del generador de manera que este ejerza un torque al motor DC. Se muestra a continuación el circuito ensamblado.

La parte más importante del circuito es el transistor BJT modelo TIP41C, ya que, este componente será el que disipe la mayor parte de la potencia eléctrica generada por el generador, por lo tanto, debe estar montado con un disipador térmico de buen tamaño. Además, se muestra el esquemático de la carga electrónica en la siguiente figura 6.

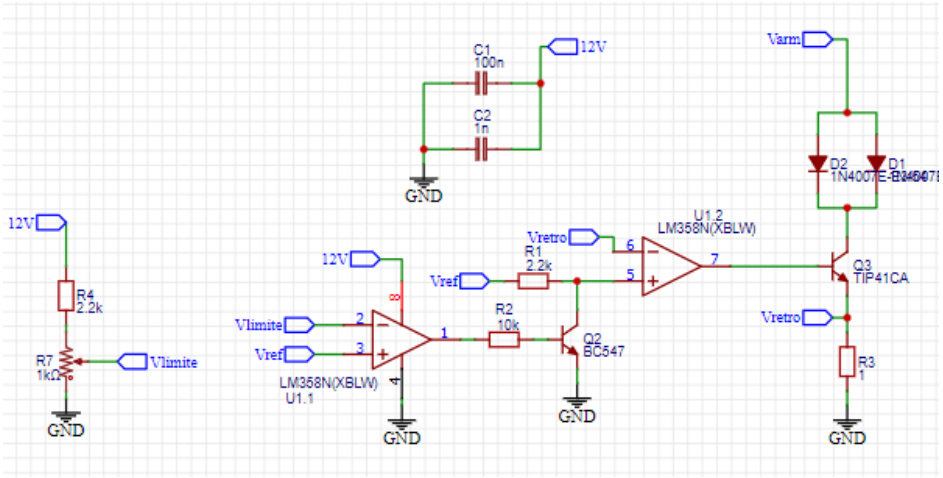

Figura 6: Esquemático carga electrónica

Para realizar la función, el circuito se alimenta con 12 Volts, y el transistor se alimenta directamente de la armadura del motor, para el control de la corriente, el módulo recibe un voltaje de entrada, este voltaje de entrada es aplicado al Opamp (amplificador operacional), y este a su vez, lo aplica sobre la resistencia R3 del esquemático de la figura 6, de esta manera, se produce una corriente igual a $\frac{Vcontrol}{R3}$, en muchas ocasiones se puede dejar R3 = 1 para que la corriente sea igual a Vcontrol, ademas, el transistor TIP41C es el encargado de proveer toda la corriente que pasar por la resistencia, por ende, el opamp termina siendo un controlador que no disipa mucha potencia (es importante que sea el modelo LM358). Por ultimo, la carga cuenta con 2 protecciones, 1) Un par de diodos D2 y D1 en la parte superior que impiden que se queme el transistor si es que se conecta en polaridad inversa la carga, y 2) el opamp 1.1 que impide que el voltaje Vcontrol aplicado a la resistencia R3 sea muy grande evitando que la carga pida mucha corriente, el valor de limite de corriente se puede ajustar a mano con el potenciómetro R7. Para garantizar un buen desempeño, se realizó una serie de pruebas a distintos voltajes y corrientes un con sensor de temperatura en el disipador para monitorear la temperatura del transistor TIP41C. Ver la tabla 2. Datos de la prueba: Tiempo de funcionamiento: 1 minuto, Temperatura ambiente: 28 grados celsius.

Tabla 2: Temperaturas medidas

V, I	0.2 A	0.4 A	0.6 A	0.8 A
5 V	28	29	31	30
9 V	29	31	33	34
12 V	32	33	38	37

Referencias

[1] Opto 22, "Snap-aiv-4: Snap 4-ch -10vdc to +10vdc analog input module." https://www.opto22.com/products/snap-aiv-4, 2025. Accedido el 23 de diciembre de 2025.

[2] Opto 22, "Snap-aov-27: Snap 2-ch -10 to +10vdc bipolar analog voltage output module." https://www.opto22.com/products/snap-aov-27, 2025. Accedido el 23 de diciembre de 2025.